

Four tools. What each one actually shows.

The shortlist, described from real interfaces: the projects' own screenshots, their documentation and a running AI Observer. For each tool: what you see, how our goals map onto it, and what is missing.

	Tokens	Cost	Attribution	Timeline	Compute	Outcome
AI Observer Telemetry dashboard for coding agents	~	✓	~	~	✗	~
OpenLIT OpenTelemetry platform with a coding-agent section	✓	✓	✓	~	~	~
Langfuse LLM tracing with timelines and agent graphs	✓	✓	✓	~	✗	~
AgentSight System-side view of agents through eBPF	~	✗	~	~	✓	✗

✓

 standard view

~

 custom query or manual

✗

 data missing

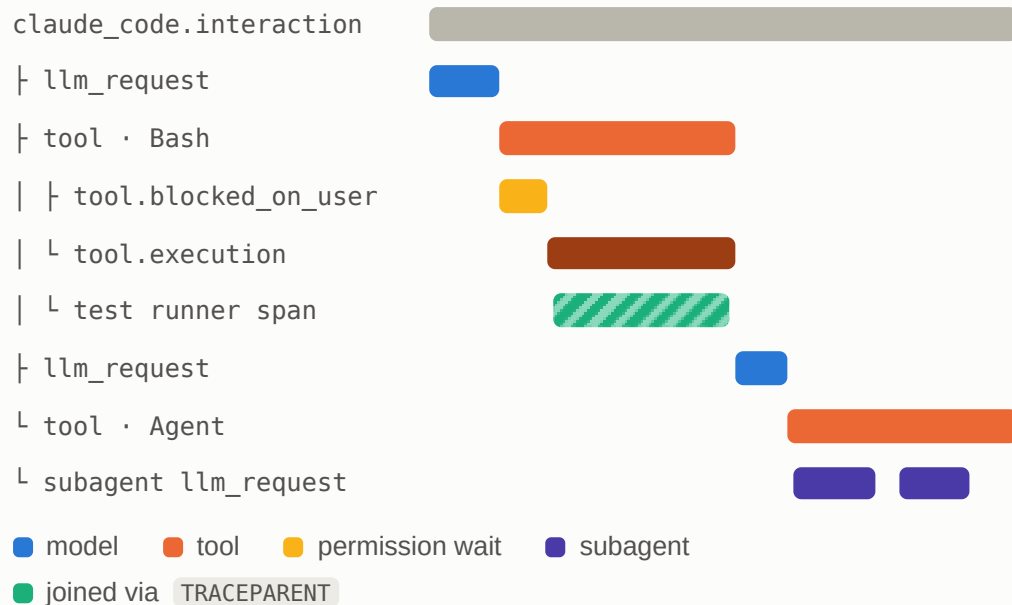
Can a person see the waste without reading raw logs?

S1 Rerun without a change Tests or other commands rerun with no code or environment change. compute time	S2 Expensive waiting A lead agent keeps calling its model while another agent works. tokens	S3 Polling Frequent process polls or status messages without new information. tokens time	S4 Long wait A wait or blocker on the path to delivery. time	S5 Spend, no result High resource use without a successful outcome. tokens compute time outcome
--	---	---	--	---

Ordinary environments Codex, Claude Code and OpenCode in a plain devcontainer or terminal, without Subyard.	Tokens, counted once Input, cached input, output and reasoning per model; cached is part of input.	Compute and time CPU, memory and processes, including test runs; a timeline of work, waits and blockers.	Attribution From task to agent or subagent to the step or tool call. Cost estimate and outcome are should-haves.
---	--	--	--

Every rating in this deck uses one scale:  visible in the tool's standard views ·  needs a custom query or manual correlation ·  the data is missing.

The agents already emit most of the model side.



Span names from the Claude Code monitoring docs. Bar lengths are illustrative, not measured.

Claude Code

- Traces (beta) with `CLAUDE_CODE_ENHANCED_TELEMETRY_BETA=1`; subagent spans carry `agent_id` and `parent_agent_id`.
- Bash subprocesses get `TRACEPARENT`, so a test runner can join the trace.
- Metric `token.usage` by type: input, cacheRead, cacheCreation, output.

Codex CLI

- Opt-in `[otel]`: events for API requests, prompts, tool approvals and results.
- Metric `turn.token_usage`: input, cached_input, output, reasoning_output.

OpenCode

- Telemetry through plugins: third-party OTLP plugin, or Langfuse's.

IMPLICATION

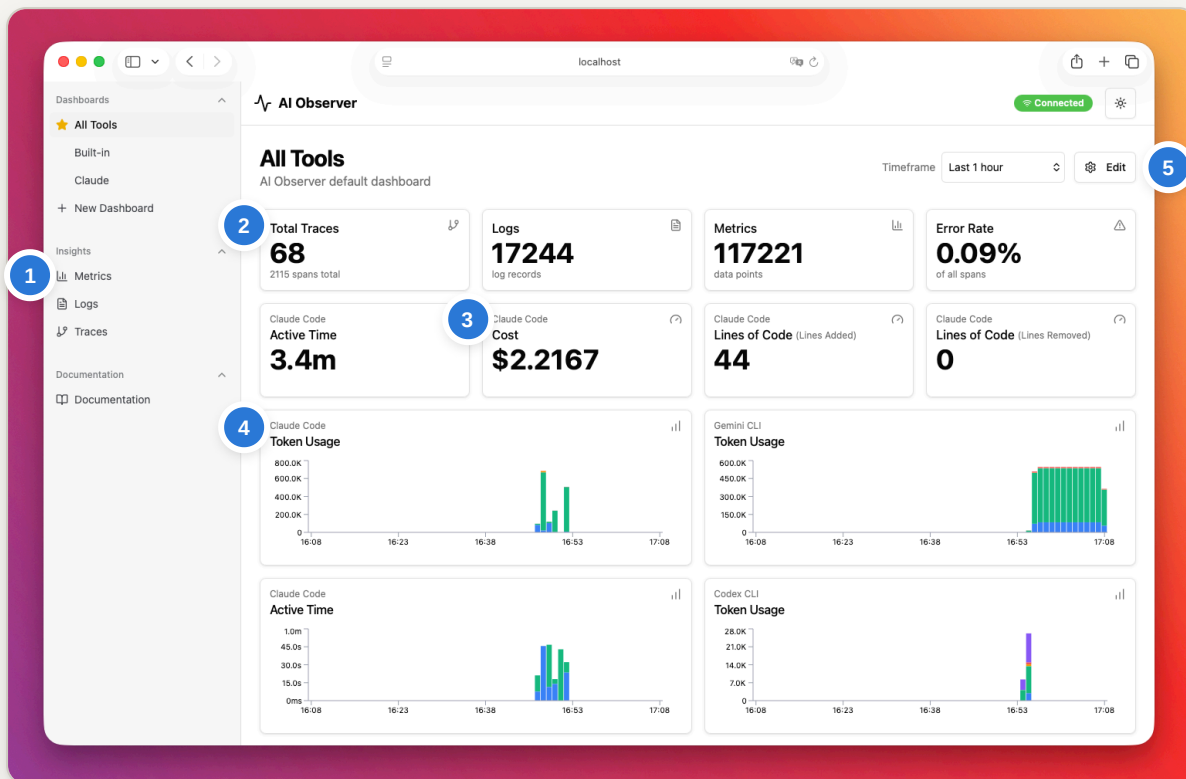
Configuration covers tokens, cost and tool calls. None of the agents reports CPU, memory or the processes it starts.

The data source decides what a tool can see.

	AGENT OTLP metrics, logs, traces	SESSION FILES ~/.claude, ~/.codex	AGENT HOOKS plugins, Stop hook	KERNEL EBPF TLS, exec, files	HOST AND PROCESSES /proc, collector	WHAT IT CAN SEE
AI Observer	● serve mode	● watch mode	—	—	—	Model calls, tokens, cost, tool calls. The two modes exclude each other.
OpenLIT	●	● token deltas	● openlit coding install	—	● separate collector	Turns, tools, subagents, Git outcome; host and process metrics beside them.
Langfuse	● traces only	● read by plugins	● Claude Code, Codex, OpenCode	—	—	The agent's own steps in detail; nothing outside the agent.
AgentSight	— exports only	● without root	—	● root or CAP_BPF	●	Processes, files, LLM traffic and CPU of any command; not the agent's intent.

PATTERN Three tools look from inside the agent. Only the system side sees test runs, CPU and memory.

A telemetry dashboard for coding agents.



Dashboard. Official screenshot from the AI Observer repository.

- 1 **Navigation**
Dashboards, Metrics, Logs, Traces, Sessions, built-in docs.
- 2 **Built-in tiles**
Traces, logs, metric points, error rate, active services.
- 3 **Metric Value widget**
One number per service: active time, cost, lines of code.
- 4 **Metric Chart widget**
Token usage over time, stacked by Cache Creation, Cache Read, Input and Output.
- 5 **Edit**
Add, move and remove widgets; several saved dashboards.
- + **Sessions page**
Agent, start, duration, model, message count. A transcript opens as Chat or Timeline: a chronological list of prompts, tool calls and results.
- + **Traces page**
Span waterfall, for example Codex `run_task` → `run_turn` → `reasoning`. Empty without OTLP traces.

Seen on a running instance: every page was checked on 0.6.0 in Subyard (watch mode, Claude Code and Codex). Its screenshots are not published because they contain session data.

Tokens and cost by agent. Blind to processes.

S1 RERUN



Bash calls are listed in the transcript, but there is no CPU, memory or process data.

S2 WAITING



Lead and subagent are separate sessions with no link between them.

S3 POLLING



Counts of tool calls per session; repetition is found by reading.

S4 LONG WAIT



Timestamps and session duration; no view of gaps. Waterfall needs OTLP traces.

S5 NO RESULT



Cost per session; outcome only through proxies: commits, PRs, lines changed.

~ Tokens

Input, cache read and write, output per agent. No per-model table.

✓ Cost

Embedded prices for Claude, OpenAI, Gemini, Copilot.

✗ Compute

CPU and memory only as Gemini CLI's own metrics.

~ Attribution

Session → message → tool call. No subagent tree.

✓ Agents

Claude Code, Codex, Gemini, Copilot; OpenCode via a third-party plugin.

✓ Setup

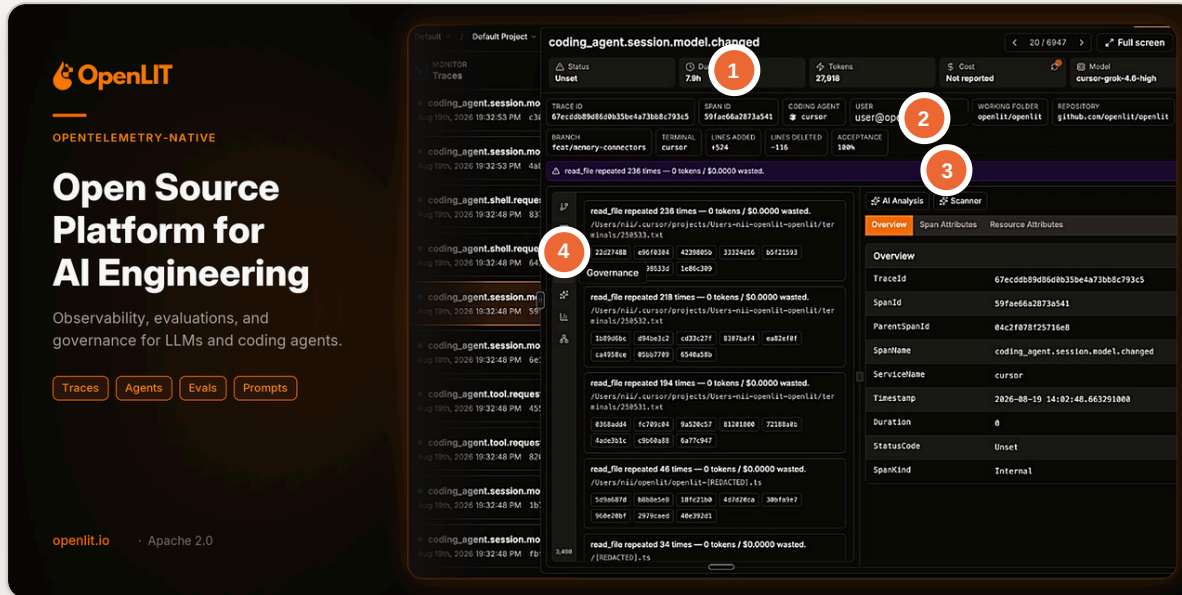
One binary or container, local DuckDB. Already in Subyard.



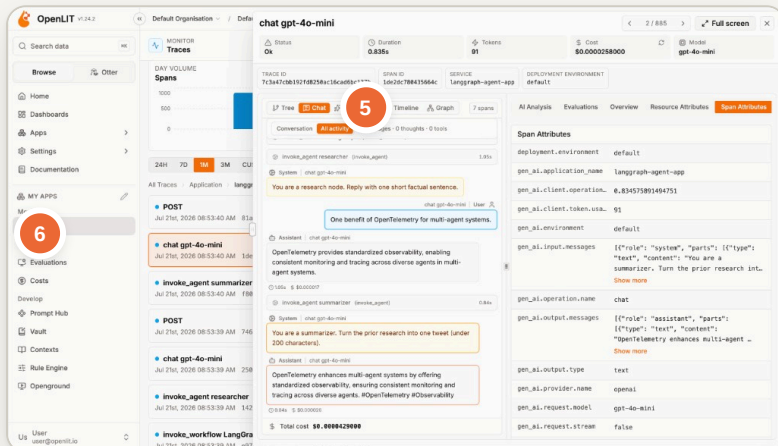
ROLE

A light token and cost baseline. On its own it cannot show S1 or S2.

Coding-agent sessions with loop badges.



Coding-agent trace. Official README image.



Trace detail. Official docs image.

- 1 **Run header**
Status, duration, tokens, cost and model of the session.
 - 2 **Coding context**
Agent, user, working folder, repository, branch, terminal, lines added and deleted.
 - 3 **Loop banner**
“read_file repeated 236 times”, with wasted tokens and cost. Same tool, same arguments.
 - 4 **Governance panel**
Each repeated call group with its spans.
 - 5 **Five views of a trace**
Tree, Chat, AI Analysis, Timeline (waterfall over elapsed time) and Graph (DAG).
 - 6 **Navigation**
Telemetry, Agents, Evaluations, Costs, Prompt Hub, Vault, Rule Engine.
- + **Coding Agents → Sessions**
Table with duration, model, tool calls, code changed, commits, PRs, tokens, cost and Outcome (merged, committed, abandoned, cancelled).

From documentation. Not run yet.

Closest to our scenarios. Compute lives next door.

S1 RERUN



The same test command twice gets a Loop badge. It compares arguments, not code; CPU is in a separate collector.

S2 WAITING



Subagent turns fold under the spawning chat; the timeline shows overlap. No “waiting” view.

S3 POLLING



Repeated identical calls are flagged in the list and on the session row, with tokens and cost wasted.

S4 LONG WAIT



Wide spans in the timeline waterfall; waits are not labelled.

S5 NO RESULT



Cost and Outcome sit in the same table. Outcome is Git-based, not test results.

✓ Tokens

Per turn: input, output, cached, reasoning.

✓ Cost

Configurable price table, separate from billing.

~ Compute

Collector exports host and per-process CPU and memory, without a GPU too. Not joined to sessions.

✓ Attribution

Session → subagent (parent id) → tool span.

~ Agents

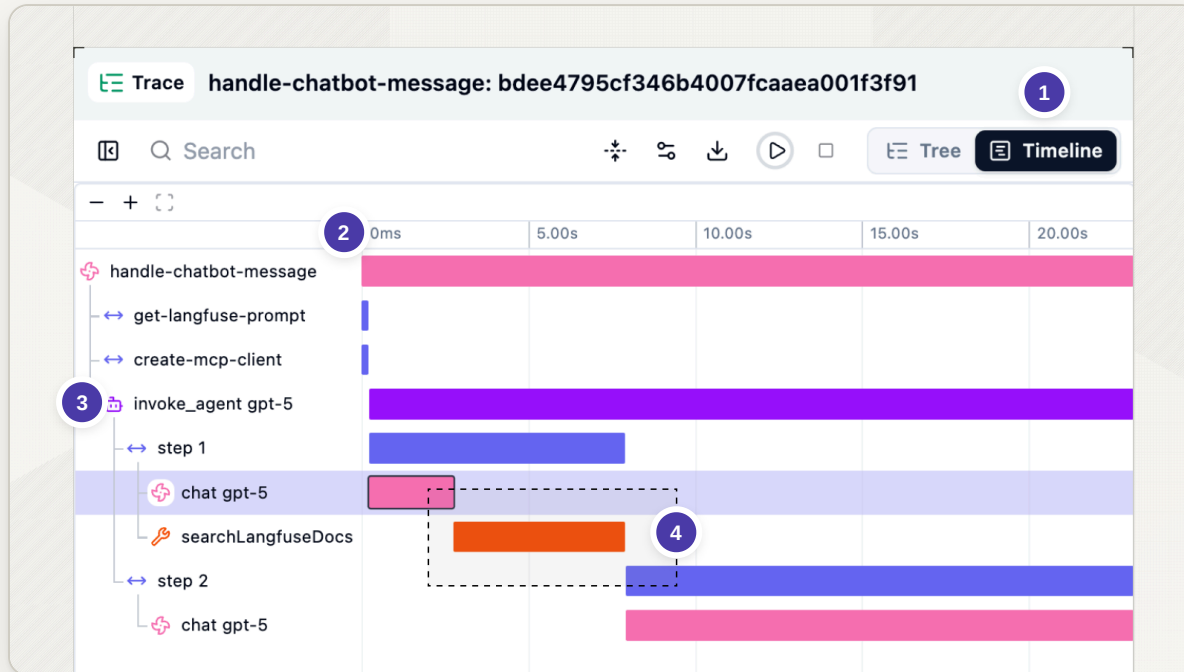
Claude Code, Codex, Cursor, Windsurf. No OpenCode.

✓ Setup

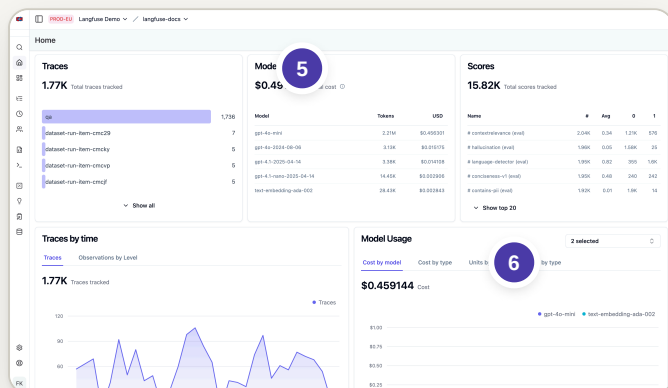
OpenLIT and ClickHouse containers plus a CLI. Content capture defaults to `full`.

 **ROLE** Strongest agent-side fit on paper. Needs the process collector and one query to tie compute to a session.

Traces you can read as a timeline.



Trace timeline, August 2026. Official image.



Home dashboard. Official docs image.

1 Tree or Timeline

Two views of one trace; the timeline zooms and pans from a dozen to over a thousand spans.

2 Elapsed-time axis

Bar width is the duration of each observation.

3 Nesting

Agent → step → model call or tool; colour marks the type.

4 Tool call

A long tool call is a wide bar: the blocker is visible at a glance.

5 Model costs

Tokens and USD per model.

6 Model usage

Cost or units by model or by usage type.

+ Agent graph

Aggregated mode merges repeated steps into one node with a counter and draws loops as cycles.

+ Observations table

Default view in v4: every span with model, tokens, cost and latency; full-text search.

From documentation. Not run yet.

Best view of the agent's steps. Nothing outside it.

S1 RERUN



Each test run is a tool span with its duration and command. No CPU, no code-change check.

S2 WAITING



Codex subagents nest under the spawning turn; repeated wait calls show as a counter.

S3 POLLING



The aggregated graph shows “step (n)”; no “no new information” check.

S4 LONG WAIT



A long tool call is a wide bar. Waits between turns fall outside any trace.

S5 NO RESULT



Cost per trace and session; success needs a score that we define.

✓ Tokens

Usage types in separate buckets. Codex plugin: cached and reasoning.

✓ Cost

Model price table with overrides.

✗ Compute

Elapsed time per span only. No CPU, memory or processes.

✓ Attribution

Session → trace → generation → tool; subagents nested.

✓ Agents

Plugins for Claude Code, Codex and OpenCode.

~ Setup

Six services: web, worker, Postgres, ClickHouse, Redis, S3.

■ **ROLE** Strong for S4 and attribution across all three agents. Needs a system-side partner for compute.

Processes, files and LLM calls in one place.

```

1 agentsight top mode=live sessions state=running refresh=2s rows=10/10
sort cpu view all filter known agents session tokens 1029.5m

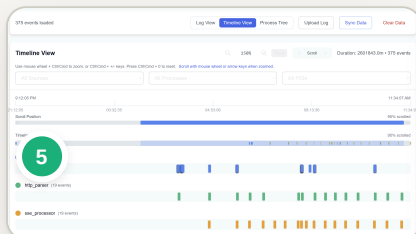
sessions
2 3 4
SESSION AGENT STATE AGE MODEL TOKENS HEALTH ACTIVITY
> codex:rollou.0d codex live 42m gpt-5.5 63.3M 398.6% 35.4G 7 proc, 714 tool, 10 file
codex:rollou.cf codex live 6h gpt-5.5 661.4M 7.5% 326M 4 proc, 6517 tool, 12 file
codex:rollou.e6 codex live 12h gpt-5.5 55.1M 1.6% 396M 4 proc, 544 tool, 10 file
claude:2f7b50.a claude live 11h claude-opus-4 4.1M 0.9% 661M 4 proc, 94 tool, 1 file
claude:d1fa99.a claude live 3h claude-opus-4 118.6M 0.9% 513M 1 proc, 348 tool, 37 file
claude:81d7b9.8 claude live 40m claude-opus-4 55.4k 0.6% 475M 3 proc, 1 tool

selected session - all
session: codex:rollou.0d3c5
agent: codex state=live pid=2307241
model: gpt-5.5

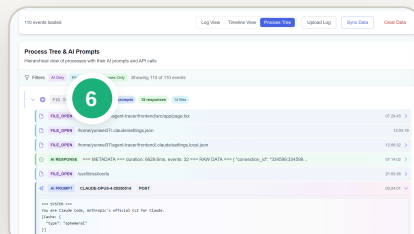
status
keys q quit | up/down select | s sort | v view | p pause | r refresh | +/- rows | ? help
session rows include agent-native local logs from ~/.claude or ~/.codex when present
proc evidence uses /proc for CPU/RSS/process families

```

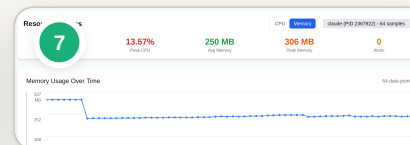
agentsight top , terminal. Official image.



Timeline



Process tree



Metrics

- 1 Live sessions, sorted by CPU**
Refreshes every 2 s; works without root from `/proc` and session files.
- 2 Tokens**
Per session, with agent and model.
- 3 CPU and memory**
For the whole process family of the agent.
- 4 Activity**
Processes, tool calls and files touched.
- 5 Timeline**
One lane per source: processes, HTTP, model streams.
- 6 Process tree**
Each process with its prompts, responses and file opens.
- 7 Metrics**
Average and peak CPU and memory per process over time.

From documentation. Not run yet. `record -- <command>` saves a run to SQLite; `report` summarises it. The local tool is open source; a hosted app is in preview.

The only one that sees CPU. Weak on meaning.

S1 RERUN



Each test process shows with CPU and memory under the agent that started it. No code-change check.

S2 WAITING



LLM traffic next to a running child on the timeline; links between agents are not modelled.

S3 POLLING



README names repeated model and tool calls and friction signals; how they show is not documented.

S4 LONG WAIT



Gaps between events on the timeline; no wait label.

S5 NO RESULT



Tokens and CPU are there; the outcome is not recorded.

~ Tokens

Input and output; cached and reasoning not documented.

✗ Cost

No price calculation documented.

✓ Compute

CPU and memory per process, process tree, files, network.

~ Attribution

Process → child process, not agent → subagent → tool.

✓ Agents

Any command; README names Claude Code, Codex, OpenCode.

~ Setup

Linux only. eBPF needs root or `CAP_BPF`; in Docker, a privileged container with the host PID namespace.

■ **ROLE** A compute partner for one of the agent-side tools, where root is available.

No single tool covers it. Pairs might.

	AI Observer	OpenLIT	Langfuse	AgentSight
SCENARIOS				
S1 Rerun without a change	✗ no process data	~ same-args loop	~ span duration	~ CPU per run
S2 Expensive waiting	~ separate sessions	~ timeline overlap	~ nested subagents	~ process timeline
S3 Polling	~ read transcript	✓ loop badge	~ graph counter	~ undocumented
S4 Long wait	~ timestamps	~ waterfall	✓ timeline	~ event gaps
S5 Spend without a result	~ Git proxies	~ Git outcome	~ own scores	✗ no outcome
REQUIREMENTS				
Tokens without double counting	~ per agent	✓ per turn	✓ per call	~ in / out
Compute	✗	~ separate collector	✗	✓ per process
Attribution	~ no subagents	✓	✓	~ process tree
Cost estimate	✓	✓	✓	✗
Codex, Claude Code, OpenCode	✓	~ no OpenCode	✓	✓
Plain devcontainer	✓ one binary	✓ two containers	~ six services	~ needs root

✓ standard view ~ custom query or manual correlation ✗ data missing AI Observer partly seen on a running instance; the rest is from documentation.

Other ways in, by the gap they fill.

Local log readers

ccusage · Sniffly

`ccusage claude|codex|opencode daily`
prints token and cost tables from local session files, no server. Sniffly is a local web dashboard for Claude Code logs.

tokens

cost

Native OTel into a generic stack

Grafana LGTM · SigNoz

Send the agents' own OTLP to a general backend. Grafana Cloud has Claude Code and Codex integrations. S1–S5 dashboards are ours to build.

tokens

timeline

LLM gateway

LiteLLM · Portkey · Helicone

A proxy behind `ANTHROPIC_BASE_URL` logs tokens, cost and latency per request. It sees requests, not tools, subagents or processes.

tokens

cost

Other tracing platforms

W&B Weave · Opik · Phoenix · Laminar

Weave has a Claude Code plugin that traces turns, tool calls and subagents. Same class as Langfuse: no compute.

tokens

timeline

Process and container metrics

cAdvisor · Tetragon · Netdata

cAdvisor: CPU and memory per container.
Tetragon: eBPF exec and exit events with the parent chain, so a test run traces back to the agent. Netdata: processes and cgroups.

compute

Reruns visible at the source

pytest cache · Nx · Turborepo · Bazel

Test and build caches key results on inputs. A cache hit, or a forced rerun, is a direct S1 signal that no observability tool infers today.

S1

Agent side plus system side. What stays open.

OpenLIT + its process collector

A. One UI, one database

Sessions with loop badges, outcome and cost; host and per-process CPU and memory in the same ClickHouse.

Open: tying process metrics to a session is a custom SQL dashboard. No OpenCode.

Langfuse + AgentSight

B. Best of each side

Langfuse for the agent tree and timeline across all three agents; AgentSight for test processes, CPU and memory.

Open: two UIs correlated by time; eBPF needs root; Langfuse is heavy to host.

Native OTel + SigNoz or Grafana + cAdvisor or Tetragon

C. Configuration only

Claude Code traces and Codex OTel into a generic backend, with container or process metrics for compute.

Open: every S1–S5 view is a dashboard we write and maintain.

ccusage

D. A cross-check

Zero setup. Daily and per-session token and cost tables to validate the totals of whichever tool we pick.

Open: tokens and cost only.

NOT COVERED BY ANY

“No code or environment change since the last run” and “waiting for another agent” are not detected out of the box.

Pick finalists. Try them on real runs.

AGENT SIDE · PROPOSAL

OpenLIT

Coding-agent sessions, loop detection, outcome column, and a process collector in the same stack.

SYSTEM SIDE · PROPOSAL

AgentSight

The only candidate that ties CPU, memory and test processes to an agent.

FALLBACK

Langfuse

If OpenLIT's Claude Code, Codex or OpenCode coverage falls short.

01

Install

In an ordinary devcontainer next to Codex, Claude Code and OpenCode, without Subyard.

02

Use in real work

Everyday tasks, not synthetic ones (decision D7).

03

Note what was quick to see

For each weak spot S1–S5: which view showed it, and how long it took.

04

Record

Fill the ratings in `solutions/`, marked “tried”; log the choice in `docs/decisions.md`.